Reviewer Pack — Arithmetic Progression of Lattice Points in Unit Square Boun…

Entry 0e33b3addd6d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 01:52:35 UTC

Arithmetic Progression of Lattice Points in Unit Square Bounds Resolution Proof Length

Entry ID: `0e33b3addd6d` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-29 01:52:35 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Arithmetic Progressions) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

> ['For a satisfiable CNF instance F, the number of arithmetic progressions of length at least 3 with common difference greater than 2 that can be formed using literals in F is bounded by a function of the resolution proof length t*(F), specifically, E[|P(F)|] ≤ α* log(t*(F)) for some absolute constant α.', 'Here, |P(F)| denotes the count of such arithmetic progressions in F.', 'If there exists an instance F with |P(F)| > α* log(t*(F)), then the conjecture is refuted.']

Rationale (proposer's reasoning):

> ['Arithmetic progressions can provide a measure of regularity or symmetry in the distribution of literals, which may correlate with the complexity of constructing resolution proofs.', 'The counting of arithmetic progressions might expose structural properties of CNF instances that are not immediately apparent through traditional measures like clause-to-literal ratios.']

Taxonomy category: `arithmetic_progressions` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `56390bde4bfca76b`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a CNF instance F with resolution proof length t*(F), if |P(F)| is greater than α* log(t*(F)) *for any n ≤ 40 and m clauses, or if any seed produces a metric exceeding α* log(t*(F)), the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=1.0s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n, m):
        cnf = []
        for _ in range(m):
            clause = [random.randint(1, n), -random.randint(1, n)]
            cnf.append(clause)
        return cnf

    def resolution_proof_length(cnf):
        # Simplified DPLL solver to estimate proof length
        clauses = set(tuple(sorted(c)) for c in cnf)
        unit_clauses = {c[0] for c in clauses if len(c) == 1}
        while unit_clauses:
            new_unit_clauses = set()
            for clause in clauses:
                if any(abs(lit) not in unit_clauses for lit in clause):
                    continue
                new_lit = -sum(lit for lit in clause if abs(lit) in unit_clauses)
                if new_lit < 0:
                    new_unit_clauses.add(-new_lit)
            if not new_unit_clauses:
                break
            unit_clauses.update(new_unit_clauses)
        return len(clauses) + len(unit_clauses)

    def count_arithmetic_progressions(cnf):
        n = max(abs(lit) for clause in cnf for lit in clause)
```

```
        progressions = set()
        for i in range(1, n+1):
            for j in range(i+1, n+1):
                diff = j - i
                if diff <= 2:
                    continue
                progression = {i, j}
                k = j + diff
                while k <= n:
                    progression.add(k)
                    k += diff
                if len(progression) >= 3:
                    progressions.add(tuple(sorted(progression)))
        return len(progressions)

    n = random.randint(5, 40)
    m = random.randint(n, n*2)
    cnf = generate_cnf(n, m)
    t_F = resolution_proof_length(cnf)
    P_F = count_arithmetic_progressions(cnf)

    if P_F > 10 * math.log(t_F):
        return {
            "metric_name": "P(F)",
            "metric_value": P_F,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": f"Instance with n={n}, m={m} has |P(F)| > 10 * log(t*(F))"
        }

    return {
        "metric_name": "P(F)",
        "metric_value": P_F,
        "instances_tested": 1,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 97) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
```

```
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='Instance with |P(F)| > 10 * log(t*(F))' first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
terexample': ''}
TRIAL: {'metric_name': 'P(F)', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': True, 'c
TRIAL: {'metric_name': 'P(F)', 'metric_value': 100, 'instances_tested': 1, 'conjecture_holds': False
TRIAL: {'metric_name': 'P(F)', 'metric_value': 81, 'instances_tested': 1, 'conjecture_holds': False,
TRIAL: {'metric_name': 'P(F)', 'metric_value': 4, 'instances_tested': 1, 'conjecture_holds': True, 'c
TRIAL: {'metric_name': 'P(F)', 'metric_value': 272, 'instances_tested': 1, 'conjecture_holds': False
TRIAL: {'metric_name': 'P(F)', 'metric_value': 30, 'instances_tested': 1, 'conjecture_holds': True, '
TRIAL: {'metric_name': 'P(F)', 'metric_value': 240, 'instances_tested': 1, 'conjecture_holds': False
TRIAL: {'metric_name': 'P(F)', 'metric_value': 49, 'instances_tested': 1, 'conjecture_holds': False,
TRIAL: {'metric_name': 'P(F)', 'metric_value': 36, 'instances_tested': 1, 'conjecture_holds': False,

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1b834650.py", line 104, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1b834650.py", line 104, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                              ~^^^^^^^^
KeyError: 'seed'
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Safety rail: critic_challenge_falsified | original: The test results show that there are instances where |P(F)| > α * log(t*(F)), which refutes the conjecture. | next: Investigate the specific instances and resolution proof lengths to understand the conditions under which the conjecture fails.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11543
2	preregistration	ollama_remote	glm4:latest	0	0	6029

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
3	novelty	ollama_remote	glm4:latest	0	0	4827
4	novelty	ollama_remote	glm4:latest	0	0	5198
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16710
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11195
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11259
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14405
9	judge	ollama_remote	glm4:latest	0	0	10315

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 91481 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0e33b3addd6d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0e33b3addd6d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/0e33b3addd6d.tar.gz` (if generated)